VNR VJIET Name of the Experiment:

Name of Laboratory: _____

_____ Date: _____

WEEK – 4

Data Pre-Processing Methods

1. Removing outliers / missing values.
2. Inputing standard values
3. Capping of values

```
# Import all the packages
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from statistics import median
import numpy as np
```

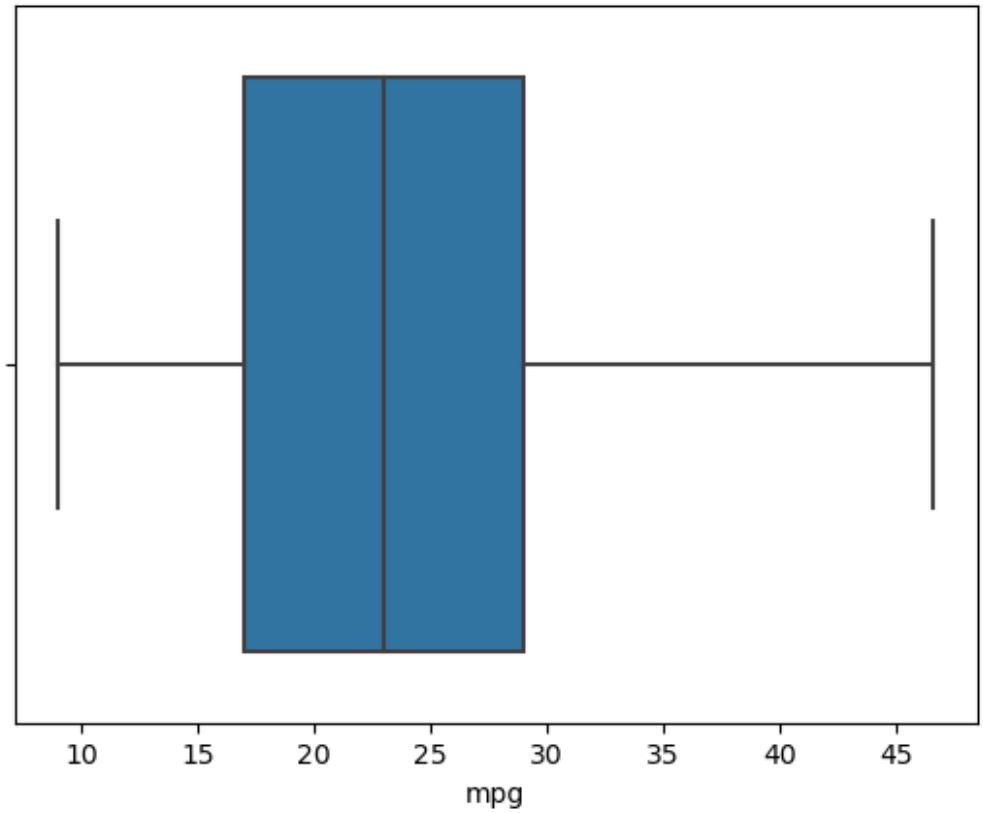
```
df = pd.read_csv('mpg_data.csv')
df.head(5)
```

```
    mpg  cylinders  displacement horsepower  weight  acceleration  model y
ear  \
0  15.0          8         350.0        165    3693          11.5
 70
1  18.0          8         318.0        150    3436          11.0
 70
2  16.0          8         304.0        150    3433          12.0
 70
3  17.0          8         302.0        140    3449          10.5
 70
4  15.0          8         429.0        198    4341          10.0
 70

   origin                car name
0       1    "buick skylark 320"
1       1   "plymouth satellite"
2       1        "amc rebel sst"
3       1          "ford torino"
4       1     "ford galaxie 500"
```

```
sns.boxplot(x=df['mpg'], data=df)
```

```
<AxesSubplot:xlabel='mpg'>
```




```
df.loc[df['horsepower'] == '?']
```

```
      mpg  cylinders  displacement horsepower  weight  acceleration  \
31   25.0          4          98.0          ?    2046          19.0
125  21.0          6         200.0          ?    2875          17.0

     model year  origin         car name
31           71       1     "ford pinto"
125          74       1  "ford maverick"
```

```
df.isnull().sum()
```

```
mpg             0
cylinders       0
displacement    0
horsepower      0
weight          0
acceleration    0
model year      0
origin          0
car name        0
dtype: int64
```

```
# replace '?' with NaN
df.replace('?', np.nan, inplace=True)
# df.loc[df['horsepower'] == '?'] , shows that '?' is replaced
```

```
df.isnull().sum()
```

```
mpg             0
cylinders       0
displacement    0
```

```
horsepower      2
weight          0
acceleration    0
model year      0
origin          0
car name        0
dtype: int64
```

1. Outlier detection and removal

The IQR method is a simple and effective way to remove outliers from a dataset. It works by identifying data points that fall outside of the range of Q1 - 1.5 * IQR and Q3 + 1.5 * IQR, where Q1 and Q3 are the first and third quartiles of the data.

```
# First and Third Quartile
q1 = df['mpg'].quantile(0.25)
q3 = df['mpg'].quantile(0.75)

# IQR
IQR = q3 - q1

# calculate lower limit and upper limit

LL = q1 - 1.5*IQR
UL = q3 + 1.5*IQR

print(q1, q3, IQR, LL, UL)
```

```
17.0 29.0 12.0 -1.0 47.0
```

```
# Identifying outliers
outliers = df[(df['mpg'] <= LL) | (df['mpg'] >= UL)]

# Remove Outliers
df_outliers_removed = df.drop(outliers.index)
```

The Z-score method is another common method for removing outliers. It works by calculating the number of standard deviations each data point is away from the mean. Data points with Z-scores greater than a certain threshold (typically 3 or 4) are considered outliers.

```
# Calculate the Z-score
z_scores = (df['mpg'] - df['mpg'].mean()) / df['mpg'].std()

# Identify outliers
outliers = df[np.abs(z_scores) > 3]

# Remove outliers
df_outliers_removed = df.drop(outliers.index)

# Drop rows with missing values
df_na_rows_removed = df.dropna()

# Drop columns with missing values
df_na_columns_removed = df.dropna(axis=1)
```

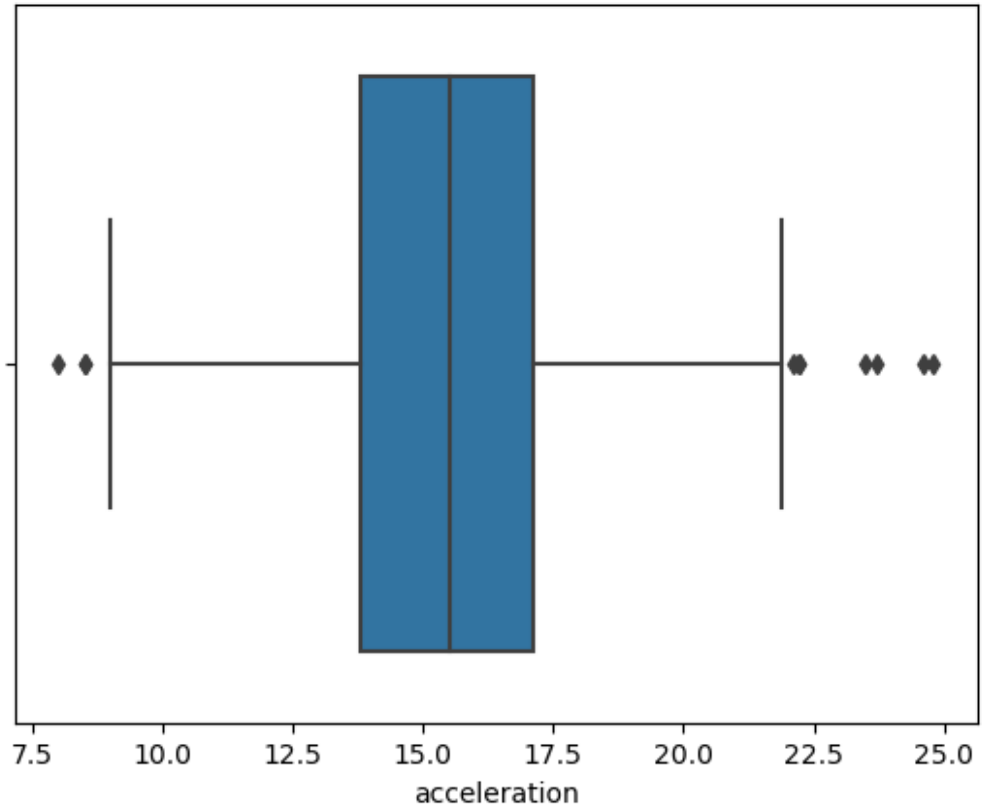
```
# Impute missing values with the median value of the column
df['mpg'].fillna(df['mpg'].median(), inplace=True)
```

2. Inputing standard values

```
# Box plot before replacing outliers with median

sns.boxplot(x='acceleration', data=df)
```

```
<AxesSubplot:xlabel='acceleration'>
```




```
# quartiles for acceleration
q1 = df['acceleration'].quantile(0.25)
q3 = df['acceleration'].quantile(0.75)
median = df['acceleration'].quantile(0.50)

IQR = q3 - q1

# calculating Lower limit and upper limit
LL = q1 - 1.5*IQR
UL = q3 + 1.5*IQR

# Outliers
outliers_acc = df[(df['acceleration'] <= LL) | (df['acceleration'] >= UL)]
outliers_acc
```

```
      mpg  cylinders  displacement horsepower  weight  acceleration  \
6    14.0          8         440.0        215    4312           8.5
8    15.0          8         390.0        190    3850           8.5
10   14.0          8         340.0        160    3609           8.0
58   23.0          4          97.0         54    2254          23.5
194  29.0          4          85.0         52    2035          22.2
```

```
195  24.5          4          98.0         60    2164          22.1
298  27.2          4         141.0         71    3190          24.8
299  23.9          8         260.0         90    3420          22.2
325  43.4          4          90.0         48    2335          23.7
389  44.0          4          97.0         52    2130          24.6

     model year  origin                            car name
6            70       1                  "plymouth fury iii"
8            70       1                 "amc ambassador dpl"
10           70       1                 "plymouth 'cuda 340"
58           72       2                  "volkswagen type 3"
194          76       1                 "chevrolet chevette"
195          76       1                    "chevrolet woody"
298          79       2                        "peugeot 504"
299          79       1  "oldsmobile cutlass salon brougham"
325          80       2                 "vw dasher (diesel)"
389          82       2                          "vw pickup"
```

replacing outliers with median values

```
# indexing method.
# Convert target values into array
arr = df['acceleration'].values

# if arr values are in between 11 and ul, represent that
# value as True else as False
# False values are outliers we can find it at 5th position
true_index = (arr>LL) & (arr < UL)
true_index
# Outliers are represented as False
```

```
array([ True,  True,  True,  True,  True,  True, False,  True, False,
        True, False,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True, False,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True, False, False,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
```

```
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True, False, False,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True, False,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True,  True,  True,  True,  True,  True,  True,  True,
        True,  True, False,  True,  True,  True])
```

```
# let us substitute mean value in the place of outliers
# so ca1culate mean of all values other than the outliers.
mid1=np.mean(df['acceleration'][true_index])
mid1
15.501542416452427
#replace False index value by mean
false_index=~true_index # ~ is bool operator for inverse
df['acceleration'].values[false_index]=mid1
df
```

```
      mpg  cylinders  displacement horsepower  weight  acceleration  \
0    15.0          8         350.0        165    3693     11.500000
1    18.0          8         318.0        150    3436     11.000000
2    16.0          8         304.0        150    3433     12.000000
3    17.0          8         302.0        140    3449     10.500000
4    15.0          8         429.0        198    4341     10.000000
..    ...        ...           ...        ...     ...           ...
388  27.0          4         140.0         86    2790     15.600000
389  44.0          4          97.0         52    2130     15.478068
390  32.0          4         135.0         84    2295     11.600000
391  28.0          4         120.0         79    2625     18.600000
392  31.0          4         119.0         82    2720     19.400000

     model year  origin                  car name
0            70       1   "buick skylark 320"
1            70       1  "plymouth satellite"
2            70       1       "amc rebel sst"
3            70       1         "ford torino"
4            70       1    "ford galaxie 500"
..          ...     ...                   ...
388          82       1     "ford mustang gl"
389          82       2           "vw pickup"
390          82       1       "dodge rampage"
391          82       1         "ford ranger"
392          82       1          "chevy s-10"

[393 rows x 9 columns]
```

3. Capping of values

There are a number of different ways to cap values. A common approach is to set a maximum and minimum value for the column. Any values that fall outside of this range are replaced with the maximum or minimum value, respectively.

Another approach is to use a percentile-based capping method. This method sets the maximum and minimum values for the column to the percentiles of the data. For example, you could set the maximum value to the 95th percentile, and the minimum value to the 5th percentile. This would remove any values that fall outside of the normal range of the data.

```
df_cap = pd.read_csv('mpg_data.csv')
df_cap.shape
```

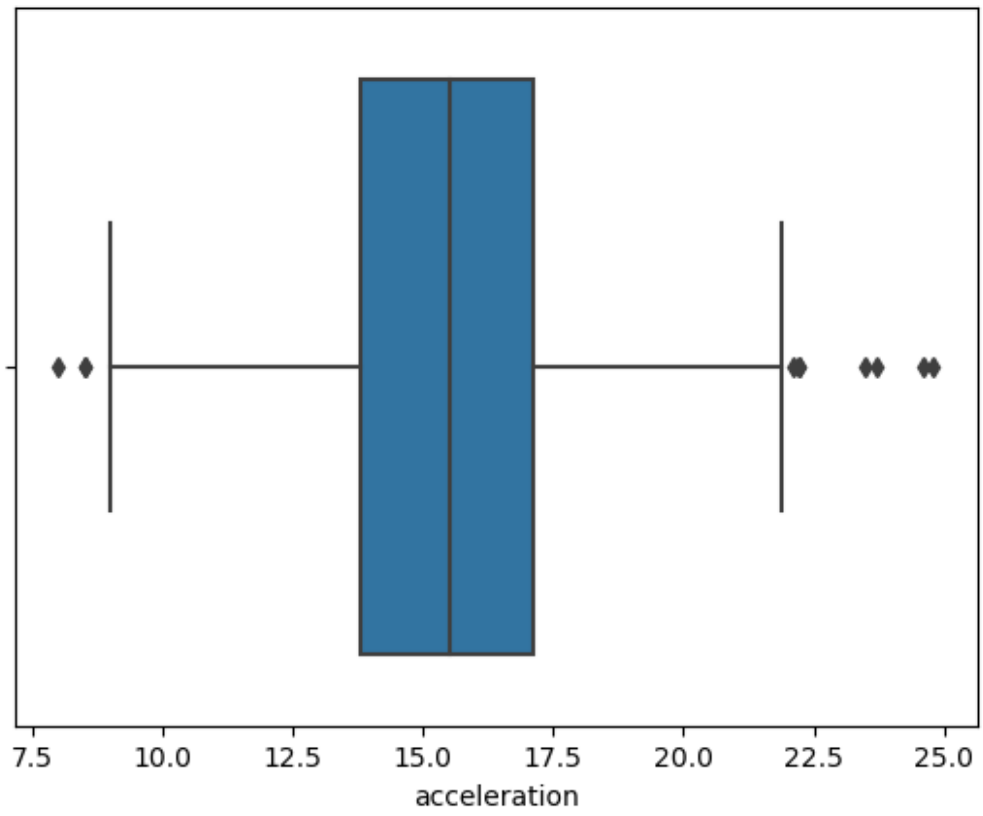
```
(393, 9)
```

```
# Identifying Outliers in cylinders
max_threshold = df_cap['acceleration'].quantile(0.95)
min_threshold = df_cap['acceleration'].quantile(0.05)
print(max_threshold, min_threshold)
```

```
20.21999999999999 11.26
```

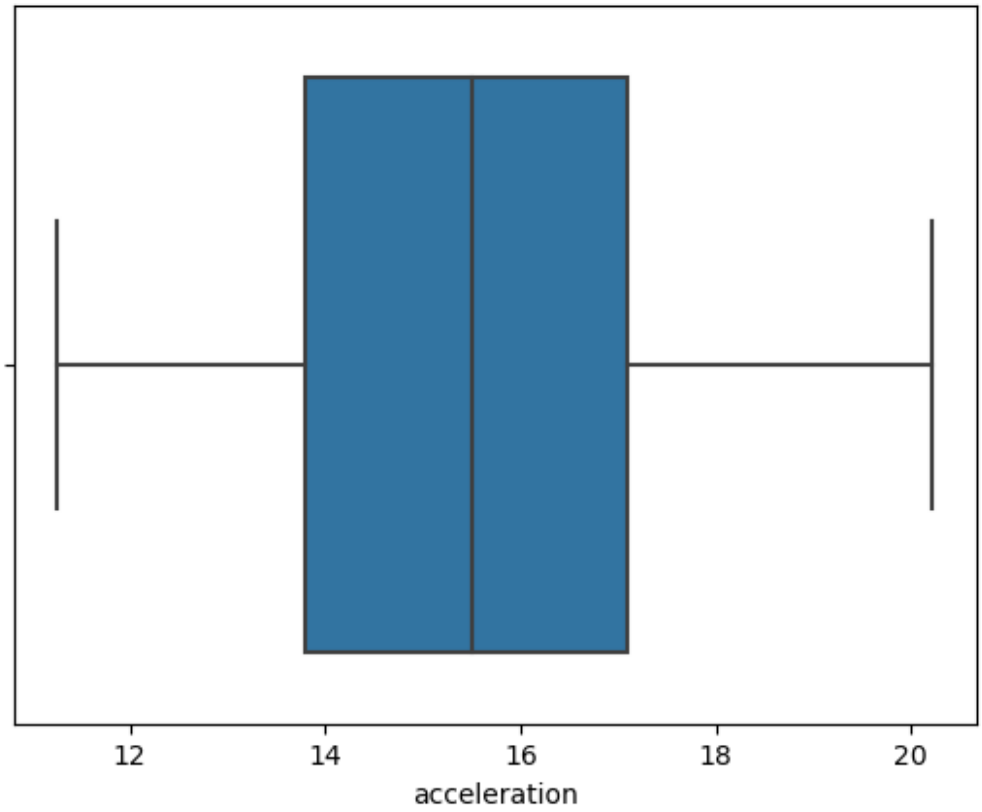
```
# Before capping, box plot
sns.boxplot(x='acceleration', data=df_cap)
```

```
<AxesSubplot:xlabel='acceleration'>
```




```
# Or
# Cap the 'price' column at min - max threshold
df_cap['acceleration'] = df_cap['acceleration'].apply(lambda x: np.clip(x,
```

```
df_cap['acceleration'].quantile(0.05), df_cap['acceleration'].quantile(0.9
5)))

# After capping, box plot
sns.boxplot(x='acceleration', data=df_cap)
```

```
<AxesSubplot:xlabel='acceleration'>
```